Bài 9: Dự án — Quản lý sản phẩm

1. Mô tả yêu cầu dự án

Dự án tổng hợp kiến thức OOP đã học: xây dựng chương trình console quản lý danh sách sản phẩm với các chức năng cơ bản: thêm sản phẩm, hiển thị danh sách, tìm kiếm theo tên, sửa thông tin, xóa sản phẩm.

2. Thiết kế class Product

Class Product áp dụng encapsulation: các thuộc tính (id, tên, giá, số lượng tồn kho) khai báo private, cung cấp constructor nhận đầy đủ thông tin, cùng getter/setter cho từng thuộc tính. Nên override phương thức toString() để in thông tin sản phẩm dễ đọc khi hiển thị danh sách, thay vì để Java in địa chỉ bộ nhớ mặc định.

3. Tách class Model và Service

Theo nguyên tắc tách trách nhiệm (separation of concerns), nên tách riêng: class Product (model, chỉ chứa dữ liệu và getter/setter), class ProductService (chứa logic nghiệp vụ: thêm, xóa, tìm kiếm, dùng ArrayList<Product> làm kho lưu trữ trong bộ nhớ), và class Main (chỉ chịu trách nhiệm hiển thị menu, đọc lựa chọn người dùng qua Scanner, gọi tới ProductService tương ứng). Cách tách này giúp mã dễ đọc, dễ mở rộng và dễ viết unit test cho từng phần riêng biệt.

4. Các chức năng CRUD cơ bản

Create: tạo đối tượng Product mới từ dữ liệu người dùng nhập, thêm vào ArrayList bằng add(). Read: duyệt ArrayList bằng vòng lặp for-each để hiển thị toàn bộ, hoặc dùng vòng lặp kết hợp so sánh tên (dùng phương thức equalsIgnoreCase hoặc contains của String) để tìm kiếm. Update: tìm sản phẩm theo id, gọi các setter tương ứng để cập nhật giá trị mới. Delete: dùng removeIf() với một điều kiện lambda, hoặc lặp tìm chỉ số rồi gọi remove(chỉ_số).

5. Xử lý ngoại lệ đầu vào

Áp dụng kiến thức try-catch đã học: bọc phần đọc dữ liệu số (giá, số lượng) bằng try-catch để bắt NumberFormatException khi người dùng nhập sai định dạng (ví dụ nhập chữ thay vì số), thông báo lỗi thân thiện và cho nhập lại thay vì để chương trình dừng đột ngột.

6. Mở rộng: lưu dữ liệu ra file

Phiên bản mở rộng có thể lưu danh sách sản phẩm ra file văn bản hoặc file CSV bằng các lớp trong package java.io (FileWriter để ghi, BufferedReader/Scanner để đọc lại khi khởi động chương trình), giúp dữ liệu không bị mất khi tắt chương trình — đây là bước đệm quan trọng trước khi học kết nối cơ sở dữ liệu thực sự ở các khóa nâng cao hơn.

Học thêm, bài tập và hướng dẫn giải

Java — Học thêm về thiết kế đối tượng và quản lý sản phẩm

Tài liệu nối tiếp các bài cú pháp, lớp, đa hình, ngoại lệ và collection. Ví dụ dùng cú pháp Java phổ biến, không yêu cầu preview feature.

Mục tiêu học thêm

- Phân biệt so sánh tham chiếu và so sánh nội dung
- Chọn collection theo thao tác cần thực hiện
- Kiểm tra dữ liệu trước khi cập nhật đối tượng và xử lý ngoại lệ có ý nghĩa

1. Tham chiếu và giá trị

Với đối tượng, == so sánh tham chiếu, còn equals thường dùng để so sánh nội dung theo cách lớp định nghĩa. Với String, dùng equals thay vì dựa vào việc các chuỗi có thể dùng chung vùng nhớ.

```
String a = new String("SV01");
String b = new String("SV01");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

Nếu biến có thể null, dùng Objects.equals(a, b) hoặc gọi equals từ chuỗi hằng. Với lớp tự viết dùng làm khóa HashMap, equals và hashCode phải nhất quán: hai đối tượng bằng nhau phải có cùng hashCode. Không thay đổi trường dùng làm khóa khi đối tượng đang nằm trong map.

2. Đóng gói và điều kiện hợp lệ

Đóng gói không chỉ là chuyển trường sang private rồi tạo setter. Setter phải bảo vệ điều kiện hợp lệ của đối tượng; nếu cập nhật lỗi, giá trị cũ phải còn nguyên.

```
class Product {
    private final String id;
    private int stock;

    Product(String id, int stock) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("Mã không được rỗng");
        }
        this.id = id;
        setStock(stock);
    }

    void setStock(int newStock) {
        if (newStock < 0) {
            throw new IllegalArgumentException("Tồn kho không âm");
        }
        stock = newStock;
    }

    int getStock() { return stock; }
}
```

Mã sản phẩm không thay đổi sau khi tạo. Tồn kho được kiểm tra trước khi gán. Khi quản lý tiền, không tùy tiện dùng double cho các phép tính cần chính xác theo đơn vị tiền; có thể dùng số nguyên theo đơn vị nhỏ nhất hoặc BigDecimal với quy tắc làm tròn rõ ràng.

3. Interface, kế thừa và composition

Interface mô tả hợp đồng hành vi, ví dụ Repository có findById và save. Một triển khai lưu bằng bộ nhớ, một triển khai lưu bằng file có thể dùng cùng hợp đồng. Lớp dịch vụ không cần biết chi tiết lưu trữ.

Kế thừa phù hợp với quan hệ “là một” và lớp con giữ được kỳ vọng của lớp cha. Nếu chỉ cần dùng chức năng của một đối tượng khác, đưa đối tượng đó vào trường của lớp thường dễ bảo trì hơn kế thừa. Đa hình giúp gọi cùng phương thức qua interface nhưng mỗi triển khai xử lý khác nhau.

4. Collection và ngoại lệ

ArrayList phù hợp cho duyệt theo thứ tự và truy cập theo chỉ số; thêm/xóa ở giữa có thể cần dịch chuyển nhiều phần tử. HashMap phù hợp cho tìm theo mã với chi phí trung bình thấp nhưng không bảo đảm thứ tự duyệt. LinkedHashMap hữu ích khi cần giữ thứ tự chèn.

```
Map<String, Integer> stocks = new HashMap<>();
stocks.put("SP01", 10);
if (stocks.containsKey("SP01")) {
    int current = stocks.get("SP01");
    if (current >= 3) stocks.put("SP01", current - 3);
}
```

Đây là ví dụ đơn luồng; thao tác kiểm tra rồi cập nhật không tự an toàn khi có nhiều luồng. Không catch Exception rồi bỏ qua: người dùng sẽ tưởng thao tác thành công. Với đọc file, try-with-resources giúp đóng tài nguyên ngay cả khi có lỗi.

> **Bài tập phân mức**
>
> 1. Dễ: giải thích kết quả == và equals trong ví dụ chuỗi.
> 2. Trung bình: viết sell(id, quantity), từ chối số lượng không dương và bán quá tồn; mã không có phải báo lỗi rõ.
> 3. Khó: tách ProductRepository và ProductService, viết hai repository bộ nhớ và file; mô phỏng lỗi ghi file mà không báo bán hàng thành công.

Gợi ý lời giải và tự kiểm tra

1. Hai new tạo đối tượng khác; equals của String so nội dung.
2. Kiểm tra quantity > 0, tồn tại sản phẩm và stock >= quantity trước khi cập nhật. Thử 0, -1, đúng bằng tồn và lớn hơn tồn.
3. Service thực hiện quy tắc nghiệp vụ, repository lưu/đọc. Lỗi lưu phải được truyền hoặc chuyển thành lỗi nghiệp vụ có nguyên nhân; không nuốt ngoại lệ. Kiểm thử file mất, nội dung sai định dạng, mã trùng và đối tượng null. Khi tách trách nhiệm đúng, đổi cơ chế lưu không buộc viết lại quy tắc bán hàng.

Nguồn học tiếp

https://dev.java/learn/ ; mục Object Oriented Programming, Collections và Exceptions.